

# Testing, Backtest & Learn Workflow

A transparent sample-testing layer, metrics, research risks, evidence-vs-conclusion design, and Thinking Note structure.

**Recommended build:** Option 2 - ASK -> CLARIFY -> DEFINE -> TEST -> LEARN

Source basis: the complete 13-page assignment PDF provided by you. Page references inside these guides refer to that assignment.

# 1. Test layer: demonstrate the workflow, not a hedge-fund backtester

Option 2 explicitly allows sample data, a public dataset, or simulated/mock data. It also explicitly says you do not need a production-grade backtesting engine. Use this freedom to make the test deterministic, transparent and easy to explain.

## Recommended test modes

| Mode                            | Pros                                       | Cons                                                    | Recommendation                                 |
|---------------------------------|--------------------------------------------|---------------------------------------------------------|------------------------------------------------|
| Deterministic simulated dataset | Fast, no external dependency, reproducible | Not real evidence                                       | Best for guaranteed demo                       |
| Bundled historical CSV          | More realistic, still deterministic        | Need to source/clean data                               | Best if you already have a clean legal dataset |
| Live/free API                   | Looks dynamic                              | Rate limits, outages, licensing/data-quality complexity | Avoid unless core build is finished            |

## 2. Minimal backtest semantics

- Compute daily returns from price series.
- Trigger on the experiment entry condition, e.g. daily return  $\leq -1\%$ .
- Enter at a clearly defined timestamp, preferably next session open/close available in your dataset.
- Measure forward return after N trading sessions.
- Subtract explicit transaction cost/slippage assumption.
- Compare triggered outcomes with a baseline, e.g. unconditional N-day forward returns.
- Avoid overlapping trades or explicitly state that you allow them. Pick one rule and document it.

## Metrics worth showing

| Metric                      | Why                                                            |
|-----------------------------|----------------------------------------------------------------|
| Number of qualifying events | Shows sample size / evidence strength.                         |
| Average forward return      | Simple edge estimate.                                          |
| Median forward return       | Reduces distortion from outliers.                              |
| Win rate                    | Easy for normal users to understand, but not sufficient alone. |
| Baseline average return     | Answers “better than what?”                                    |
| Difference vs baseline      | Directly expresses conditional edge.                           |
| Worst / best outcome        | Shows distribution risk.                                       |

## Result object

```
{
```

```

    "sampleSize": 47,
    "meanForwardReturnPct": 0.62,
    "medianForwardReturnPct": 0.31,
    "winRatePct": 55.3,
    "baselineMeanReturnPct": 0.21,
    "edgeVsBaselinePct": 0.41,
    "costAdjusted": true,
    "warnings": ["Small sample size", "Simulated/sample data"]
}

```

### 3. Research risks the Thinking Note should mention

| Risk                              | How your prototype handles / acknowledges it                                            |
|-----------------------------------|-----------------------------------------------------------------------------------------|
| Ambiguous definitions             | Clarification step before testing.                                                      |
| Incorrect assumptions             | Visible provenance + confirmation.                                                      |
| Data quality                      | Dataset/source label + missing-value checks.                                            |
| Look-ahead bias                   | Explicit entry timing; no future information in trigger.                                |
| Transaction costs                 | Visible cost assumption, included in result.                                            |
| Slippage                          | Visible bps assumption.                                                                 |
| Overfitting                       | Avoid tuning threshold to maximize one dataset result; suggest out-of-sample test next. |
| Insufficient evidence             | Show event count and warning for small N.                                               |
| Multiple comparisons              | Mention that repeated hypothesis searching can create false discoveries.                |
| Survivorship / index construction | Mention if using constituent-level data; less relevant for a single index series.       |

### 4. Learn layer: evidence vs interpretation

#### What the data shows

Use factual statements generated directly from calculated metrics: “47 events matched the rule. The average 5-day forward return was 0.62% after the configured costs, versus a 0.21% baseline.”

#### What we can reasonably conclude

Use cautious wording: “In this sample, the condition was associated with a higher average forward return than the baseline. This does not establish that the rule will remain profitable in future markets.”

#### What to investigate next

- Repeat across different date ranges.
- Compare 0.5%, 1.0%, and 1.5% thresholds without cherry-picking.
- Split high-volatility vs low-volatility regimes.
- Test next-open execution and higher slippage.
- Use an out-of-sample period.

### 5. Thinking Note outline - maximum 2 pages

| Section               | Content                                                                                                |
|-----------------------|--------------------------------------------------------------------------------------------------------|
| Interpretation        | Define what “sharp fall” could mean; distinguish qualitative phrase from testable threshold.           |
| Known vs assumed      | Table of User said / Assumed / Ask user.                                                               |
| Minimum questions     | Threshold, exit/holding period, entry timing, test period, costs.                                      |
| Experiment definition | Instrument, trigger, entry, exit, test period, filters, costs, hypothesis.                             |
| What can go wrong     | Bias, costs, data quality, overfitting, small sample, misleading conclusions.                          |
| Design philosophy     | Transparent assumptions; deterministic evidence; AI for structure/explanation, not fabricated results. |

**The strongest “Learn” screen explicitly admits limitations. That is not weakness - it directly matches the assignment’s request to recognize ambiguity and misleading conclusions.**

# Confirmed Implementation Stack

This page supersedes any earlier wording that presented the AI provider or database as optional choices. For this submission, the selected implementation is fixed:

| Layer              | Selected technology  | How it is used                                                                                 |
|--------------------|----------------------|------------------------------------------------------------------------------------------------|
| AI / LLM           | Google Gemini API    | Parse natural-language research questions, identify ambiguity, propose clarification questions |
| Database           | Neon PostgreSQL      | Persist research sessions, structured experiments, clarification answers, test runs, and con   |
| ORM                | Drizzle ORM          | Typed schema, migrations, and database access from the Next.js server layer.                   |
| Validation         | Zod                  | Validate both user input and Gemini structured output before persistence or testing.           |
| Frontend / Backend | Next.js + TypeScript | App Router UI plus server-side route handlers/server functions.                                |

## Environment variables

```
GEMINI_API_KEY=...
DATABASE_URL=postgresql://...
GEMINI_MODEL=gemini-2.5-flash
```

## Recommended data flow

Question -> server validation -> Gemini structured interpretation -> Zod validation -> clarification/confirmation -> experiment persisted to Neon -> deterministic TypeScript test -> result persisted -> Learn view.

## Security rules

Never expose GEMINI\_API\_KEY or DATABASE\_URL in client code. Keep both outside NEXT\_PUBLIC variables. Validate request sizes, parse Gemini output against a strict schema, use parameterized ORM queries, add API rate limiting, and never let the LLM fabricate backtest metrics.

## Suggested Neon schema

**research\_sessions:** id, original\_question, status, created\_at  
**experiments:** id, session\_id, instrument, timeframe, entry\_condition, exit\_condition, holding\_period, test\_period, filters\_json, costs\_json, hypothesis, provenance\_json  
**test\_runs:** id, experiment\_id, dataset\_source, metrics\_json, warnings\_json, created\_at

Why this choice is strong for the assignment: Gemini is used for language understanding and ambiguity handling, while Neon stores reproducible research state. Numerical evidence remains deterministic and auditable instead of being invented by the model.